

LAPORAN AKHIR PROYEK
Analisis Performa Model DenseNet, ResNet, EfficientNet, dan InceptionV3
pada Deteksi Hama dan Penyakit Tanaman Berbasis Gambar



Team Member:

David Goanli	2702223582
Jason Emanuel Halim	2702242410
Kelson	2702245135
Meghan Hillary Mardjohan	2702223443
Rafael Komala	2702227611

Course Code: LB01

Semester: 5

BINUS UNIVERSITY
JAKARTA
2025

ABSTRAK

Hama dan penyakit tanaman merupakan salah satu ancaman utama terhadap produktivitas dan kualitas hasil pertanian, sehingga diperlukan metode identifikasi yang cepat dan akurat untuk mendukung pengambilan keputusan di sektor pertanian. Penelitian ini bertujuan untuk membandingkan dan mengevaluasi beberapa arsitektur *deep learning pretrained model*, yaitu EfficientNet-B0, ResNet-50, DenseNet-121, dan InceptionV3 dalam melakukan klasifikasi hama dan penyakit tanaman pada gambar dengan menggunakan pendekatan *computer vision*. Dataset yang digunakan adalah Crop Pest and Disease Detection yang terdiri dari 25.220 gambar berlabel dengan 22 kelas yang telah melalui tahap *exploratory data analysis* (EDA) dan *preprocessing* untuk meningkatkan kualitas data. Proses *training* model dilakukan dengan menggunakan teknik *transfer learning* dan *fine-tuning* dengan konfigurasi *training* yang seragam. Performa model akan dievaluasi dengan menggunakan *classification report* (*accuracy*, *precision*, *recall*, dan *F1-score*) serta *confusion matrix* untuk mengidentifikasi pola kesalahan prediksi. Hasil eksperimen menunjukkan bahwa EfficientNet-B0 memberikan performa terbaik dengan *test accuracy* sebesar 0,91 dan kemampuan generalisasi yang stabil dibandingkan model lainnya. Kemudian, model terbaik tersebut diimplementasikan ke aplikasi PlantCare sebagai sistem klasifikasi hama dan penyakit tanaman agar dapat mendukung proses identifikasi dengan lebih efisien pada kondisi nyata di sektor pertanian.

I. PENDAHULUAN

1. Latar Belakang

Pertanian merupakan sektor penting yang menopang perekonomian dan ketahanan pangan di banyak negara. Namun, produktivitas tanaman sering terdampak oleh serangan hama dan penyakit yang menyebabkan kerusakan pada daun, batang, atau buah pada tanaman. Identifikasi dini terhadap hama dan penyakit tanaman diperlukan agar bisa mengendalikan hasil pertanian secara efektif. Seiring berkembangnya teknologi, sistem klasifikasi hama dan penyakit tanaman dengan pendekatan *computer vision* bisa menjadi solusi yang menjanjikan bagi pertanian modern.

2. Rumusan Masalah

Proses identifikasi hama dan penyakit tanaman secara tradisional masih bergantung pada pengamatan visual manual yang dilakukan oleh manusia. Hal tersebut memerlukan waktu yang lama dan sulit diterapkan pada skala pertanian yang luas. Oleh karena itu, permasalahan utama dalam penelitian ini adalah bagaimana cara mengembangkan sistem berbasis *computer vision* yang mampu melakukan klasifikasi hama dan penyakit tanaman secara langsung, akurat, dan efisien untuk mendukung pengelolaan pertanian yang lebih efektif.

3. Tujuan

Penelitian ini bertujuan untuk membandingkan dan mengevaluasi 4 arsitektur model *deep learning pretrained*, yaitu EfficientNet-B0, ResNet-50, DenseNet-121, dan InceptionV3 dalam melakukan tugas klasifikasi hama dan penyakit tanaman berdasarkan hasil analisis *classification report* dan *confusion matrix* untuk menilai performa masing-masing model. Selanjutnya, model dengan performa terbaik akan diimplementasikan pada aplikasi PlantCare.

4. Signifikansi

Penelitian ini memiliki signifikansi dengan menyediakan sistem klasifikasi hama dan penyakit tanaman berbasis *deep learning* yang dapat membantu proses identifikasi dengan lebih cepat dan akurat untuk mendukung pengelolaan pertanian yang lebih efektif. Secara akademis, penelitian ini memberikan wawasan komparatif mengenai performa berbagai arsitektur CNN *pretrained* yang dapat menjadi referensi

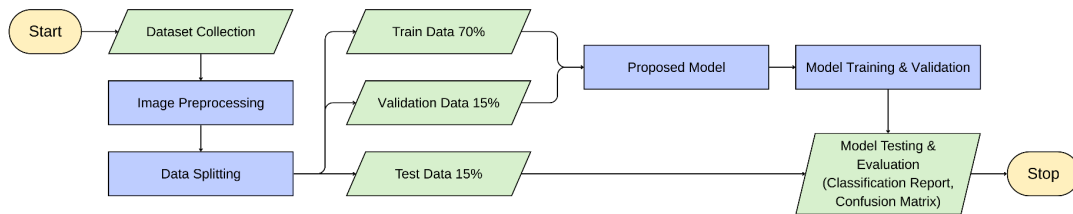
bagi penelitian selanjutnya dalam pengembangan sistem klasifikasi hama dan penyakit tanaman berbasis *computer vision*.

II. KAJIAN PUSTAKA

No.	Judul	Tujuan	Metodologi	Hasil	Limitasi
1.	Crop disease detection using deep learning techniques on images	Membandingkan performa beberapa model deep learning dalam klasifikasi penyakit tanaman berbasis gambar.	Menguji model CNN AlexNet, GoogLeNet, ResNet50, dan InceptionV3 menggunakan pendekatan transfer learning.	Model GoogLeNet mencapai akurasi tertinggi hingga 99.9% dibandingkan performa model lainnya.	Pengujian terbatas pada dataset terkontrol dan belum merepresentasikan kondisi secara nyata.
2.	Deep learning for image-based cassava disease detection	Mengembangkan sistem deteksi penyakit daun singkong berbasis gambar menggunakan deep learning yang akurat dan diterapkan pada perangkat mobile.	Menggunakan transfer learning dengan model InceptionV3 pada gambar daun singkong lapangan dan membandingkan classifier softmax, SVM, dan kNN.	Model mencapai akurasi tinggi hingga 93%, dengan performa terbaik pada penyakit CBSD, BLS, dan CMD.	Dataset terbatas pada satu wilayah dan sebagian besar gambar hanya mempresentasikan satu penyakit per daun.
3.	Technological advancements in automated crop pest and disease detection: A review & ongoing research	Mengulas kemajuan teknologi otomatisasi pendeteksian hama dan penyakit tanaman menggunakan <i>computer vision</i> dan pembelajaran mesin untuk meningkatkan akurasi deteksi di pertanian.	Studi tinjauan literatur terhadap berbagai pendekatan, termasuk CNN, IoT, segmentasi, data augmentation, dan metode deteksi hama/penyakit menggunakan gambar pertanian.	Rangkuman berbagai teknologi dan tren terbaru menunjukkan bahwa kombinasi deep learning dan sensor pintar berpotensi meningkatkan deteksi secara otomatis pada berbagai jenis tanaman.	Tidak melakukan eksperimen sendiri; hanya merangkum studi lain sehingga kontribusi empiris langsung terhadap klasifikasi gambar serangga atau dataset tertentu masih kurang.
4.	Insect pest image	Penelitian ini bertujuan	Menggunakan dua arsitektur	Model Faster R-CNN	Dataset yang digunakan

	detection and classification using deep learning	mengembangkan sistem deteksi dan klasifikasi hama tanaman menggunakan pendekatan Faster R-CNN berbasis EfficientNet untuk meningkatkan akurasi identifikasi hama dalam gambar pertanian.	Faster R-CNN dengan backbone EfficientNet-B4 dan EfficientNet-B7 pada subset kelas dari dataset IP102. GPU dipakai untuk pelatihan, data diproses melalui RPN dan ROI pooling untuk deteksi serta klasifikasi objek.	EfficientNet-B7 mencapai akurasi klasifikasi tinggi hingga sekitar 99% (5 kelas), 96% (10 kelas), dan 93% (15 kelas), mengungguli model lain yang diuji.	memiliki ketidakseimbangan kelas yang tinggi dan jumlah kelas yang terbatas pada pilihan subset, sehingga generalisasi model ke kelas yang lebih banyak belum optimal.
5.	Deep learning-based leaf disease detection in crops using images for agricultural applications	Mengembangkan sistem deteksi dini penyakit daun tanaman berbasis deep learning untuk meningkatkan kualitas hasil pertanian dan mengurangi kerugian ekonomi akibat penyakit tanaman.	Melakukan fine-tuning hiperparameter pada model CNN pre-trained, yaitu DenseNet-121, ResNet-50, VGG-16, dan Inception V4 menggunakan dataset PlantVillage.	Model DenseNet-121 menghasilkan akurasi klasifikasi tertinggi sebesar 99.81% melampaui performa model lainnya.	Penelitian tidak secara eksplisit membahas keterbatasan model, khususnya terkait pengujian pada kondisi lingkungan nyata di luar dataset terkontrol.
6.	EfficientNet: rethinking model scaling for convolutional neural networks	Mengembangkan arsitektur CNN yang efisien melalui strategi penskalaan model yang terkontrol untuk meningkatkan akurasi klasifikasi.	Menerapkan pendekatan <i>compound scaling</i> pada arsitektur EfficientNet dengan menyeimbangkan kedalaman jaringan, lebar, dan resolusi input.	EfficientNet menunjukkan performa klasifikasi yang unggul dengan kompleksitas model lebih efisien dibandingkan arsitektur CNN konvensional.	Pengujian model masih terbatas pada dataset umum sehingga performa EfficientNet pada domain spesifik belum dievaluasi secara menyeluruh.

III. METODOLOGI PENELITIAN



Gambar 1. *Methodology flowchart.*

1. Dataset Penelitian

Dataset yang digunakan dalam penelitian ini adalah *Crop Pests and Diseases dataset* yang diperoleh dari Kaggle dan di-*upload* oleh Nirmal Sankalana pada tahun 2023 dengan lisensi CC0 Public Domain. Dataset ini terdiri dari 25.220 gambar berlabel yang merepresentasikan 22 kelas hama dan penyakit tanaman pada 4 jenis tanaman pertanian utama, yaitu *cashew*, *cassava*, *maize*, dan *tomato*. Tujuan penggunaan dataset ini adalah untuk mendukung pengembangan sistem *computer vision* otomatis dalam mendeteksi hama dan penyakit tanaman pada sektor pertanian.

2. *Exploratory Data Analysis (EDA)*

a) Analisis Distribusi dan Keseimbangan Data antar Kelas

Analisis distribusi data dilakukan dengan menghitung jumlah gambar pada setiap kelas. Hasil analisis menunjukkan bahwa dataset bersifat tidak seimbang, terlihat dari kelas *Maize healthy* yang memiliki jumlah gambar paling rendah, yaitu 208 dan kelas *Tomato septoria leaf spot* yang memiliki jumlah gambar paling tinggi, yaitu 2743, selisihnya sebesar 2535 gambar.

Kelas	Jumlah Gambar
Cashew anthracnose	1729
Cashew gumosis	392
Cashew healthy	1368
Cashew leaf miner	1378
Cashew red rust	1682
Cassava bacterial blight	2614
Cassava brown spot	1481
Cassava green mite	1015
Cassava healthy	1193
Cassava mosaic	1205

Maize fall armyworm	285
Maize grasshoper	673
Maize healthy	208
Maize leaf beetle	948
Maize leaf blight	1006
Maize leaf spot	1259
Maize streak virus	979
Tomato healthy	470
Tomato leaf blight	1301
Tomato leaf curl	518
Tomato septoria leaf spot	2743
Tomato verticillium wilt	773

Tabel 1. Distribusi jumlah gambar pada setiap kelas.

b) Mendeteksi Gambar Duplikat

Deteksi gambar duplikat dilakukan untuk menghindari pengulangan data yang dapat menyebabkan bias pada proses pelatihan model. Tahap ini dilakukan melalui 2 pendekatan, yaitu mengecek duplikasi dalam kelas yang sama (*same class*) dan antar kelas yang berbeda (*cross class*), dengan menggunakan teknik *image hashing* untuk mengukur kemiripan visual antar gambar.

Kelas	Jumlah Gambar Terduplikat
Cashew red rust	1
Cassava bacterial blight	20
Cassava brown spot	14
Cassava mosaic	92
Maize fall anyworm	1
Maize leaf blight	3
Maize leaf spot	23
Tomato healthy	3
Tomato leaf blight	2
Tomato septoria leaf spot	448
Total Duplicates per Class	607

Tabel 2. Jumlah gambar duplikat pada setiap kelas.

Berdasarkan hasil analisis *same class* pada tabel 2, ditemukan sebanyak 607 gambar duplikat yang tersebar di beberapa kelas, dengan jumlah terbanyak berasal dari kelas *Tomato septoria leaf spot*. Selain itu, analisis *cross class*

mengidentifikasi adanya 224 pasangan gambar duplikat yang muncul pada dua kelas berbeda.

Cross-Class Duplicate 1



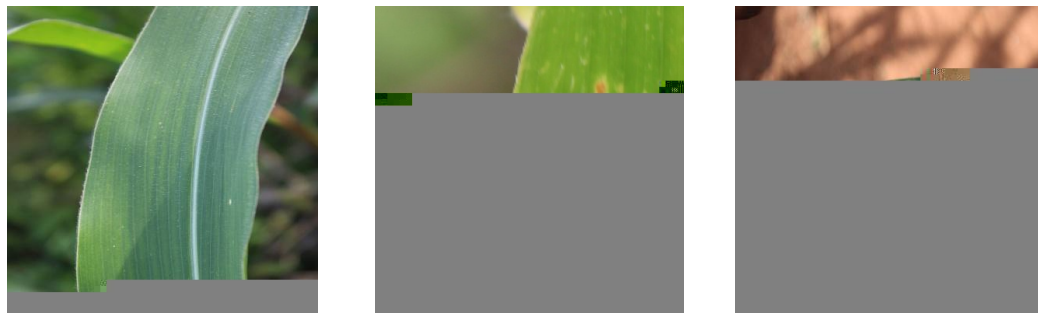
Cross-Class Duplicate 2



Gambar 2. Contoh gambar duplikat yang ditemukan pada kelas berbeda.

c) Mendeteksi Gambar Rusak

Pendeteksian gambar rusak dilakukan untuk memastikan seluruh data dapat diproses dengan baik. Gambar rusak didefinisikan sebagai gambar yang tidak dapat dibuka menggunakan *library* cv2 maupun PIL. Hasilnya terdapat 94 gambar terdeteksi rusak.

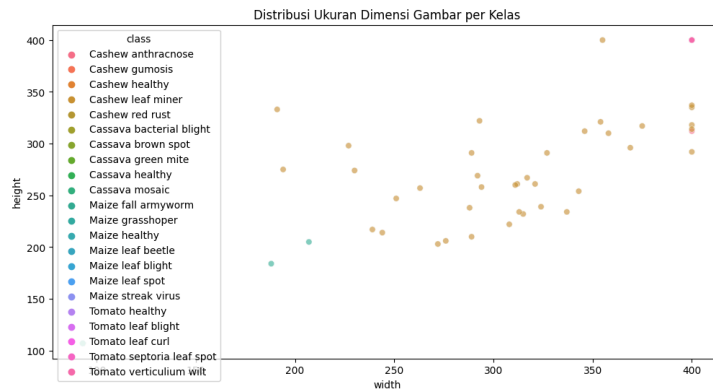


Gambar 3. Contoh gambar rusak pada dataset.

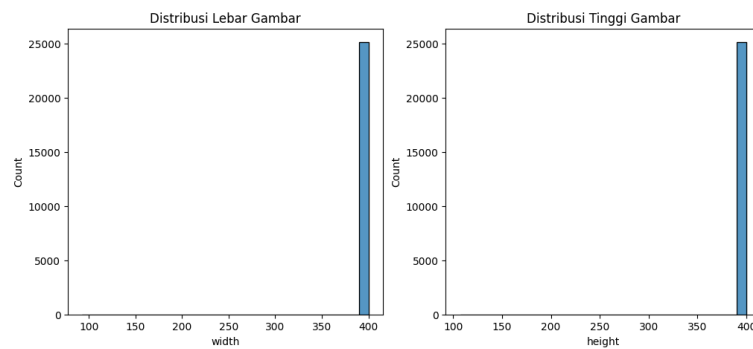
d) Analisis Dimensi per Kelas

Analisis dimensi gambar dilakukan untuk mengetahui variasi ukuran gambar dalam dataset dengan memeriksa resolusi lebar dan tinggi masing-masing

gambar. Hasilnya menunjukkan terdapat variasi ukuran gambar yang cukup signifikan dan perlu di-*resize* agar seragam sebelum diproses oleh model.



Gambar 4. Distribusi ukuran dimensi gambar per kelas.

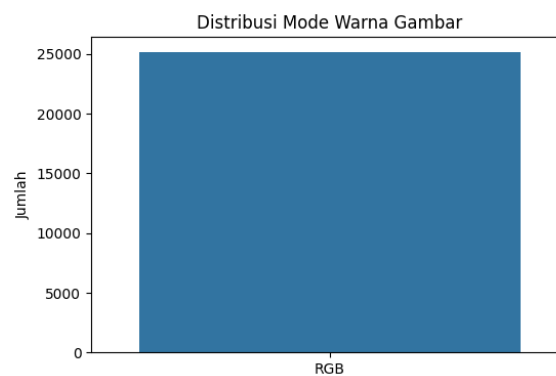


Gambar 5. Dimensi lebar dan tinggi gambar.

Gambar 4 dan 5 menunjukkan bahwa sebagian besar gambar berukuran 400×400 piksel, tapi masih ada sebagian kecil gambar dengan ukuran yang berbeda.

e) Pemeriksaan RGB

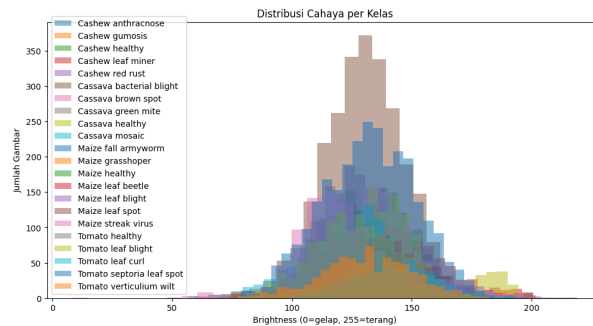
Pemeriksaan format warna perlu dilakukan untuk memastikan semua gambar menggunakan format RGB yang sesuai dengan kebutuhan model CNN. Hasilnya menunjukkan bahwa semua gambar telah dalam format RGB.



Gambar 6. Distribusi mode warna gambar dalam dataset.

f) Analisis Tingkat Cahaya Gambar

Analisis tingkat cahaya dilakukan untuk mengamati variasi pencahayaan antar kelas yang dapat mempengaruhi kualitas fitur visual. Nilai rata-rata intensitas cahaya dihitung pada setiap kelasnya untuk memastikan tidak terdapat perbedaan ekstrem yang dapat mengganggu proses pembelajaran model.

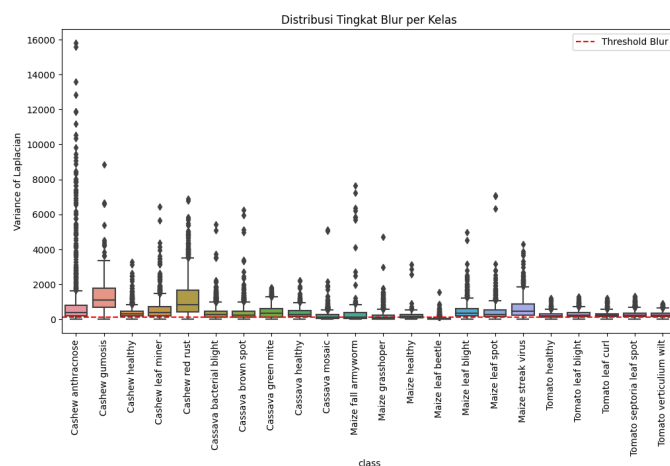


Gambar 7. Distribusi tingkat cahaya gambar per kelas.

Berdasarkan gambar 7, terlihat bahwa beberapa kelas memiliki tingkat intensitas cahaya yang lebih tinggi dibandingkan kelas lainnya, namun perbedaannya masih dalam rentang yang dapat dinormalisasi pada tahap *preprocessing*.

g) Deteksi Gambar Blur

Deteksi gambar blur dilakukan untuk mengidentifikasi gambar dengan tingkat ketajaman rendah yang berpotensi menurunkan performa model. Metode *variance of Laplacian* digunakan untuk mengukur tingkat ketajaman gambar, dimana nilai variansi yang rendah mengindikasikan bahwa gambar tersebut *blur*. Hasilnya menunjukkan bahwa beberapa kelas memiliki tingkat blur yang lebih tinggi, terutama pada kelas *Cashew anthracnose* dibandingkan kelas lainnya.



Gambar 8. Distribusi tingkat blur gambar per kelas.

3. *Preprocessing*

Setelah tahap EDA, dataset akan melalui *preprocessing* untuk meningkatkan kualitas data sebelum digunakan pada *training* model. Seluruh data hasil preprocessing akan disimpan dalam folder *dataset preprocessed*.

a) Penghapusan Gambar Rusak

Gambar yang teridentifikasi rusak pada tahap EDA dihapus dari dataset untuk mencegah *error* saat proses pelatihan. Langkah ini memastikan hanya gambar valid yang digunakan dalam proses *training* dan evaluasi model. Total gambar yang dihapus pada tahap ini sebanyak 94 gambar, dengan jumlah gambar berkurang dari 25.220 menjadi 25.126 gambar.

b) Penghapusan Gambar Duplikat

Penghapusan gambar duplikat dilakukan melalui 2 tahap, yaitu penghapusan duplikat dalam kelas yang sama (*same class*) dan antar kelas (*cross class*), dengan tujuan mengurangi redundansi data dan mencegah ambiguitas label pada dataset. Pada tahap penghapusan duplikat dalam kelas yang sama, sebanyak 607 gambar dihapus, sehingga jumlah dataset menjadi 24.519 gambar dan mengurangi pengulangan pola visual dalam kelas yang sama. Selanjutnya, pada tahap penghapusan duplikat antar kelas, teridentifikasi 224 pasangan gambar duplikat muncul pada 2 kelas berbeda, sehingga perlu dilakukan penghapusan pada kedua kelas tersebut, sehingga didapatkan 448 gambar (224×2) untuk menghindari kebingungan dalam penentuan label, dan total dataset setelah proses ini menjadi 24.071 gambar.

c) Penyesuaian Tingkat Cahaya Gambar

Penyesuaian tingkat cahaya dilakukan untuk menormalkan intensitas pencahayaan gambar sehingga perbedaan cahaya antar gambar tidak terlalu signifikan. Langkah ini membantu model dalam mengekstraksi fitur visual secara lebih konsisten.

Kelas	Rata-Rata Cahaya
Cashew anthracnose	130.97
Cashew gumosis	123.05
Cashew healthy	133.21
Cashew leaf miner	138.10
Cashew red rust	129.11
Cassava bacterial blight	130.21

Cassava brown spot	133.78
Cassava green mite	130.35
Cassava healthy	143.57
Cassava mosaic	124.87
Maize fall armyworm	129.85
Maize grasshoper	136.22
Maize healthy	120.55
Maize leaf beetle	131.59
Maize leaf blight	133.68
Maize leaf spot	127.03
Maize streak virus	123.37
Tomato healthy	133.99
Tomato leaf blight	126.80
Tomato leaf curl	143.08
Tomato septoria leaf spot	132.75
Tomato verticillium wilt	131.56

Tabel 3. Rata-rata tingkat cahaya gambar setelah penyesuaian per kelas.

Pada tabel 3, tingkat cahaya gambar antar kelas menjadi lebih seragam setelah dilakukan penyesuaian pencahayaan, sehingga model dapat mengekstraksi fitur visual secara lebih konsisten.

d) Penajaman Gambar (*Unsharp Masking*)

Penajaman gambar dilakukan menggunakan teknik *unsharp masking* yang disesuaikan berdasarkan tingkat blur gambar. Tingkat *blur* diukur dengan metode *variance of Laplacian* dengan nilai *threshold* sebesar 100, di mana gambar dengan nilai di bawah ambang tersebut akan dikategorikan sebagai gambar *blur*. Untuk gambar dengan tingkat *blur* tinggi ($\text{score} < 50$), digunakan nilai *strength* sebesar 1.0, sedangkan untuk gambar dengan tingkat *blur* lebih rendah akan menggunakan nilai *strength* sebesar 0.7. Semakin *blur* suatu gambar, maka akan semakin kuat proses penajaman yang diterapkan.

4. Pemilihan Model *Pretrained*

Dalam proyek ini, kami membandingkan 4 model pretrained yang telah menunjukkan performa yang baik pada penelitian sebelumnya tentang *crop pest and disease classification*, yaitu:

a) EfficientNet-B0

EfficientNet-B0 merupakan model CNN yang mengoptimalkan *accuracy* dan efisiensi komputasi dengan menggunakan pendekatan *compound scaling* pada *depth*, *width*, dan *resolution*. EfficientNet-B0 ini merupakan *baseline* dari keluarga EfficientNet yang dirancang lebih ringan dari versi yang lainnya. Setelah dilakukan eksperimen, EfficientNet-B0 mencapai *train accuracy* sebesar 0.9891, *train loss* 0.0304, *val accuracy* sebesar 0.9088, dan *val loss* 0.3946 pada epoch ke-20.

b) ResNet-50

ResNet-50 merupakan model CNN dengan 50 layer yang menggunakan *residual connections* untuk mengatasi masalah *vanishing gradient* pada *deep neural network*. Mekanisme ini dapat membuat aliran gradien jadi lebih stabil sehingga model bisa dilatih secara efektif walaupun arsitekturnya dalam. Setelah dilakukan eksperimen, ResNet-50 mencapai *train accuracy* sebesar 0.9609, *train loss* 0.1111, *val accuracy* sebesar 0.8930, dan *val loss* 0.3957 pada epoch ke-8.

c) InceptionV3

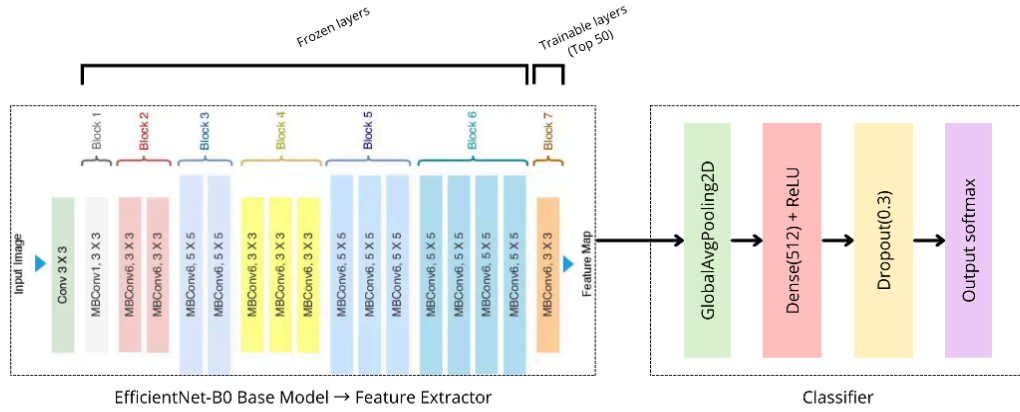
InceptionV3 merupakan arsitektur CNN yang menggunakan modul *Inception*, yaitu kombinasi *convolution* paralel berukuran 1×1 , 3×3 , dan 5×5 , untuk menangkap fitur pada berbagai skala secara efisien. Selain itu, model ini mengurangi jumlah parameter dengan teknik berupa *factorized convolutional* dan *dimensional reduction* sehingga komputasinya tetap efisien. Setelah dilakukan eksperimen, InceptionV3 mencapai *train accuracy* sebesar 0.8436, *train loss* 0.3735, *val accuracy* sebesar 0.7226, dan *val loss* 0.9003 pada epoch ke-10.

d) DenseNet-121

DenseNet-121 merupakan model CNN yang menggunakan arsitektur *dense connectivity*, artinya setiap layernya akan terhubung langsung ke seluruh layer sebelumnya. Hal tersebut memungkinkan terjadinya *feature reuse*, memperkuat aliran gradien, dan mengurangi jumlah parameternya. Setelah dilakukan eksperimen, DenseNet-121 mencapai *train accuracy* sebesar 0.9869, *train loss* 0.0578, *val accuracy* sebesar 0.8416, dan *val loss* 0.5326 pada epoch ke-16.

Berdasarkan hasil eksperimen tersebut, model pretrained EfficientNet-B0 yang dipilih untuk tahap deployment aplikasi karena performa pada *validation set*-nya yang paling bagus, yaitu *val accuracy* sebesar 0.9088 dan *val loss* 0.3946.

5. Arsitektur Model



Gambar 9. Arsitektur model EfficientNet-B0 yang diusulkan dengan *custom classifier*.

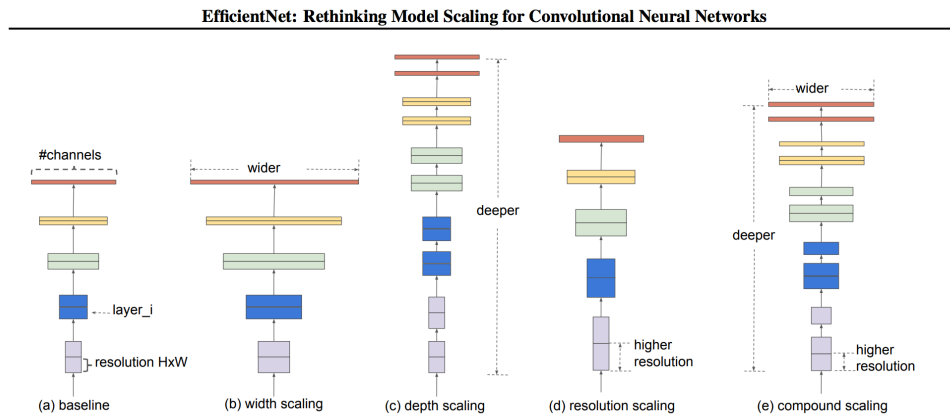
a) *Pretrained Model EfficientNet-B0 (Feature Extractor)*:

Berdasarkan arsitektur EfficientNet, *pretrained model* EfficientNet-B0 digunakan sebagai model baseline ($\phi = 0$) yang menjadi dasar penerapan prinsip *compound scaling* pada varian EfficientNet-B1 hingga EfficientNet-B7. Prinsip ini dapat dinyatakan dalam persamaan:

$$\text{depth: } d = \alpha^\phi, \text{width: } w = \beta^\phi, \text{resolution: } r = \gamma^\phi$$

dengan batasan: $\alpha \cdot \beta^2 \cdot \gamma^2 \approx 2, \alpha \geq 1, \beta \geq 1, \gamma \geq 1 \dots (1)$

dimana ϕ merupakan *compound scaling coefficient* yang mengontrol *resources* untuk *model scaling*.



Gambar 10. *Compound scaling method* dari EfficientNet-B0.

i) *Depth Scaling (d)*

Parameter *depth* digunakan untuk mengatur jumlah layer atau jumlah pengulangan blok MBConv di dalam *neural network*.

ii) *Width Scaling (w)*

Parameter *width* digunakan untuk mengontrol jumlah channel yang pada setiap *layer convolution*. Semakin besar nilai *width*, semakin banyak *feature map* yang dapat dipelajari pada setiap layer.

iii) *Resolution Scaling (r)*

Parameter *resolution* mengatur ukuran spasial input gambar yang akan masuk ke *neural network*. Resolusi yang lebih tinggi membuat model bisa menangkap detail visual yang lebih halus, tapi juga meningkatkan ukuran *feature map*-nya sehingga berdampak pada komputasi dan konsumsi memorinya.

Penelitian ini menggunakan EfficientNet-B0 sebagai model baseline ($\phi = 0$), sehingga parameter depth, width, dan resolution bernilai 1. Artinya, tidak terjadi proses scaling tambahan, hanya mengikuti konfigurasi asli dari EfficientNet-B0.

Stage	Operator	Resolution	#Channels	#Layers
1	Conv 3×3	224×224	32	1
2	MBConv1, $k=3 \times 3$	112×112	16	1
3	MBConv6, $k=3 \times 3$	112×112	24	2
4	MBConv6, $k=5 \times 5$	56×56	40	2
5	MBConv6, $k=3 \times 3$	28×28	80	3
6	MBConv6, $k=5 \times 5$	28×28	112	3
7	MBConv6, $k=5 \times 5$	14×14	192	4
8	MBConv6, $k=3 \times 3$	7×7	320	1
9	Conv 1×1 (Top Conv)	7×7	1280	1

Tabel 4. Rincian arsitektur EfficientNet-B0.

Berdasarkan struktur arsitektur EfficientNet-B0 yang ditunjukkan pada gambar 9 dan tabel 4, berikut adalah penjelasan mengenai alur pemrosesan gambar dan fungsi dari tiap layer yang ada di dalam model:

i) *Initial Convolution Layer (Conv 3×3)*

Input shape gambar berukuran $224 \times 224 \times 3$ akan diproses oleh *convolution layer* dengan *kernel* 3×3 dan *stride* 2 untuk menyiapkan representasi awal fiturnya seperti *edges*, tekstur, warna, dan sebagainya sebelum masuk ke blok inti *MBConv*. *Output shape* yang dihasilkan adalah $112 \times 112 \times 32$.

ii) *Mobile Inverted Bottleneck Convolution (MBConv)*

Bagian ini adalah blok inti dari EfficientNet-B0 yang akan digunakan berulang kali di *stage* 2 sampai 8. MBConv ini adalah pengembangan dari *inverted residual block* yang ada pada MobileNetV2, dengan penambahan *Squeeze-and-Excitation (SE) mechanism*. Terdiri dari 4 tahap pemrosesan, yaitu:

1) *Expansion (1×1 Convolution)*

Channel input diperluas (*expanded*) menggunakan convolution 1×1 dengan expansion ratio sesuai nama MBConv, misalnya akan di-*expand* $6 \times$ untuk MBConv6 stage 3, dari $112 \times 112 \times 24$ menjadi $112 \times 112 \times 144$ ukuran *feature map*-nya. Tujuannya adalah untuk meningkatkan kapasitas representasi fitur.

2) *Depthwise Convolution ($k \times k$)*

Convolution dengan *kernel* $k \times k$ dilakukan secara terpisah pada setiap *channel*, sehingga komputasinya jadi jauh lebih efisien karena jumlah parameternya hanya sebesar $k \times k \times \text{Channel}$. Hal ini bertujuan untuk menangkap pola spasial lokal pada setiap *channel*.

3) *Squeeze-and-Excitation (SE) Block*

Mekanisme ini bertugas untuk memberikan *attention* pada *channel* yang paling informatif. Pada tahap *squeeze*, *Global Average Pooling* digunakan untuk meringkas *feature map*. Contohnya pada *feature map* yang sudah diperbesar di *expansion stage*, yaitu $112 \times 112 \times 144$ akan di-*squeeze* menjadi $1 \times 1 \times 144$. Pada tahap *excitation*, vektor hasil *squeeze* akan melalui 2 *fully connected layer* dengan *activation function nonlinear* dan *sigmoid* pada layer akhir yang akan menghasilkan vektor angka berukuran $1 \times 1 \times 144$ yang berupa

bobot nilai tingkat kepentingan masing-masing *channel*. Terakhir, pada tahap *scale*, *feature map* asli ($112 \times 112 \times 144$) akan dikalikan dengan bobot dari tahap *excitation* (*weight* berukuran $1 \times 1 \times 144$) untuk memperjelas *channel* mana yang kuat dan yang lemah. *Output* dari tahap ini adalah *feature map* berbobot dengan ukuran $112 \times 112 \times 144$.

4) *Projection (1×1 Convolution)*

Feature map akan kembali dikompresi ke *output shape* yang ditentukan menggunakan *convolution* 1×1 . Contohnya, *feature map* hasil *scaling* akan $112 \times 112 \times 144$ dikompresikan kembali ke $112 \times 112 \times 24$.

iii) *Skip Connection*

Mekanisme *skip connection* ini akan diterapkan pada blok MBConv yang memenuhi 2 kondisi, yaitu *stride* = 1 dan jumlah *channel* input = jumlah *channel* output. Rumusnya dapat dinyatakan dalam persamaan, yaitu:

$$\text{Output} = X + \text{MBConv}(X) \dots (2)$$

Karena dimensi spasial dan jumlah *channel feature map*-nya tetap konsisten, maka *feature map input*-nya (X) dapat ditambahkan secara langsung ke output blok MBConv. Sehingga informasi fitur dari layer sebelumnya tetap terjaga dan dapat diteruskan ke layer berikutnya secara efektif.

Hasil akhir dari *stage* 8 pada *pretrained model* EfficientNet-B0 adalah *feature map* berukuran $7 \times 7 \times 320$ yang merepresentasikan fitur tingkat tinggi hasil ekstraksi dari seluruh proses konvolusi. Selanjutnya, akan diproses oleh top convolution layer berupa convolution 1×1 yang berfungsi untuk meningkatkan jumlah *channel* dari 320 menjadi 1280. Layer ini diikuti oleh *Batch Normalization* dan activation function *Swish* untuk menstabilkan dan memperkaya representasi fitur. Dengan demikian, output akhir *backbone* EfficientNet-B0 yang digunakan sebagai *feature extractor* adalah *feature map* berukuran $7 \times 7 \times 1280$.

Pada penelitian ini, *weight* dari *pretrained model* EfficientNet-B0 akan digunakan sebagai *feature extractor*. Sebagian besar layernya akan di-*freeze* dan 50 layer teratasnya akan di-*unfreeze* untuk proses *fine-tuning* untuk menyesuaikan hasil representasi fiturnya terhadap karakteristik dari dataset yang digunakan, yaitu *crop pest and disease classification*.

b) *Custom Classifier*

Layer	Input Shape	Output Shape
GlobalAveragePooling2D	$7 \times 7 \times 1280$	1280
Dense (512) + ReLU	1280	512
Dropout (0.3)	512	512
Output Dense (Softmax)	512	22 kelas

Tabel 5. Rincian arsitektur *custom classifier*.

i) *GlobalAveragePooling2D*

Mengubah feature map hasil *feature extraction* menjadi vektor 1 dimensi dengan mengambil rata-rata setiap channel sehingga informasi penting tetap dipertahankan.

ii) *Dense (512) + ReLU*

Menggunakan konsep *non linear* untuk mempelajari pola fitur yang lebih kompleks dan menyaring informasi yang kurang relevan.

iii) *Dropout (0.3)*

Mengurangi risiko *overfitting* dengan menonaktifkan sebagian *neuron* secara acak selama proses *training*.

iv) *Output Dense (Softmax)*

Mengonversi fitur akhir jadi probabilitas untuk setiap kelas di dataset dan mempunyai total nilai probabilitas = 1.

Model EfficientNet-B0 yang diusulkan ini memiliki total 4.716.729 parameter, dengan 3.194.022 parameter yang dilatih dan 1.522.707 parameter yang dibekukan.

6. *Training Setup*

Dataset akan dibagi menjadi 70% data *training*, 15% data *validation*, dan 15% data *testing*. Seluruh gambar akan diubah ukurannya menjadi 224×224 piksel. Sebelum proses *training*, setiap *batch* akan di-*preprocess* menggunakan *function preprocess_input()* yang sesuai dengan model yang digunakan, sehingga format dan

normalisasi gambarnya mengikuti standar ImageNet. Lalu, dataset *training* akan diacak sebanyak 1000 sampel untuk mencegah model mempelajari pola urutan data dan seluruh datasetnya akan menggunakan mekanisme *prefetch* (*AUTOTUNE*) untuk mempercepat *pipeline*-nya dan mengurangi *bottleneck* selama proses *training*.

Model akan di-*compile* menggunakan *Adam optimizer* dengan *learning rate* sebesar $1e-4$ dan *loss function* *sparse_categorical_crossentropy*. Metode *class weight* juga diterapkan untuk menangani ketidakseimbangan kelas yang ada pada dataset sehingga model tidak bias terhadap kelas tertentu. Selain itu, *callback EarlyStopping* dengan *patience* sebesar 5 digunakan untuk mencegah proses *training* berlanjut ketika performa model sudah mengalami stagnan. Tapi, pada eksperimen EfficientNet-B0, *EarlyStopping* tidak digunakan untuk menghindari kasus model berhenti *training* di *epoch* awal. Waktu training EfficientNet-B0 juga relatif singkat dibanding model lainnya, sehingga model dicoba untuk dilatih sampai seluruh *epoch*-nya selesai. Setiap model akan dilatih selama 20 *epoch* dengan *batch size* sebesar 32.

Masing-masing pretrained model juga akan di-*finetuning* dengan rincian sebagai berikut:

- a) EfficientNet-B0 akan di-*finetuning* untuk 50 layer teratas.
- b) ResNet-50 akan di-*finetuning* untuk 100 layer teratas.
- c) InceptionV3 akan di-*finetuning* untuk layer setelah 249 layer awal.
- d) DenseNet-121 akan di-*finetuning* untuk 100 layer teratas.

7. Evaluasi Model

Setiap model yang diusulkan akan dievaluasi menggunakan *classification report*, seperti metrik *accuracy*, *precision*, *recall*, dan *F1-score* untuk menilai performa model pada setiap kelas. *Confusion matrix* juga digunakan untuk mengamati jumlah prediksi yang benar dan salah pada masing-masing kelas.

IV. IMPLEMENTASI DAN HASIL

1. Spesifikasi Sistem

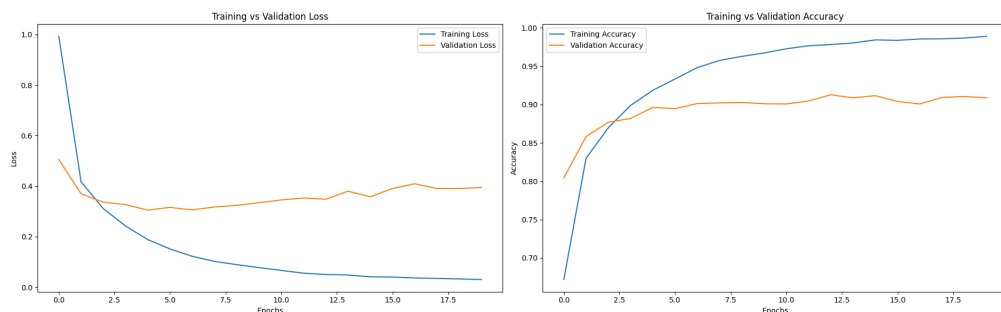
Eksperimen ini dilakukan dengan menggunakan *software Visual Studio Code*. Sedangkan *hardware* yang digunakan adalah sebuah sistem dengan prosesor Intel Core i7 generasi ke-12 (i7-12650H) berkecepatan 2.30 GHz, RAM 16GB (4800 MT/s), serta GPU dengan kapasitas VRAM sebesar 4 GB.

2. Eksperimen

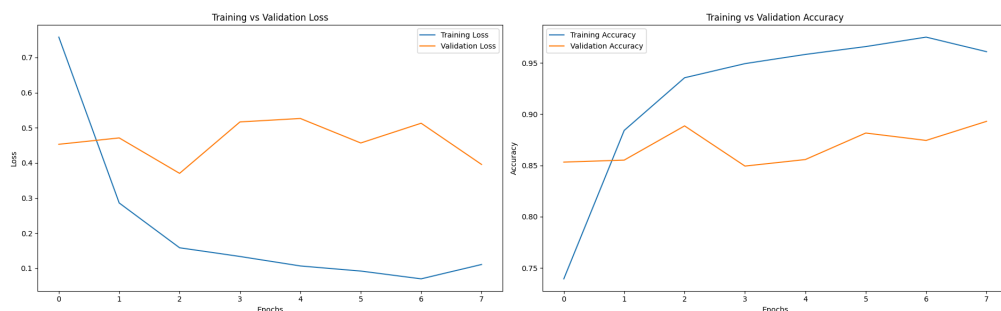
	Train Acc	Val Acc	Test Acc	Train Loss	Val Loss	Test Loss
EfficientNet-B0	0.9891	0.9088	0.9106	0.0304	0.3946	0.4045
ResNet-50	0.9609	0.8930	0.8853	0.1111	0.3957	0.3646
InceptionV3	0.8436	0.7226	0.7312	0.3735	0.9003	0.8318
DenseNet-121	0.9869	0.8416	0.8420	0.0578	0.5326	0.5354

Tabel 6. Hasil eksperimen perbandingan model EfficientNet-B0, ResNet-50, InceptionV3, dan DenseNet-121.

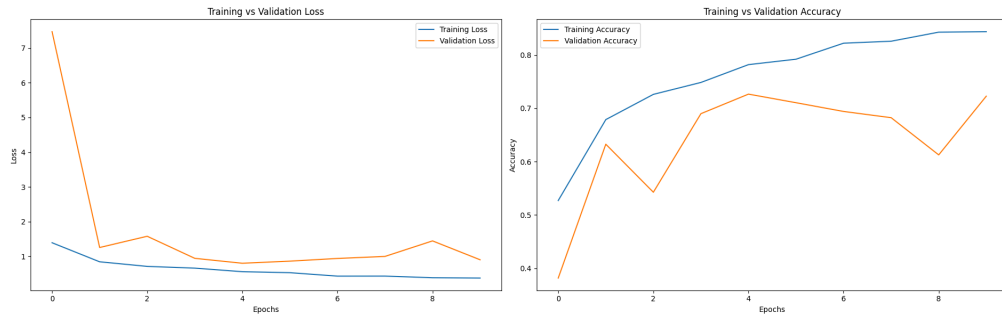
3. Visualisasi Hasil



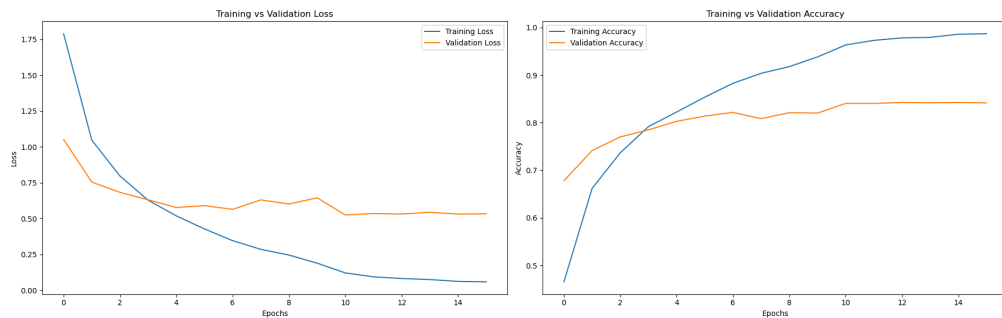
Gambar 11. Perbandingan grafik *training vs validation loss* dan *training vs validation accuracy* pada model EfficientNet-B0.



Gambar 12. Perbandingan grafik *training vs validation loss* dan *training vs validation accuracy* pada model ResNet-50.



Gambar 13. Perbandingan grafik *training vs validation loss* dan *training vs validation accuracy* pada model InceptionV3.



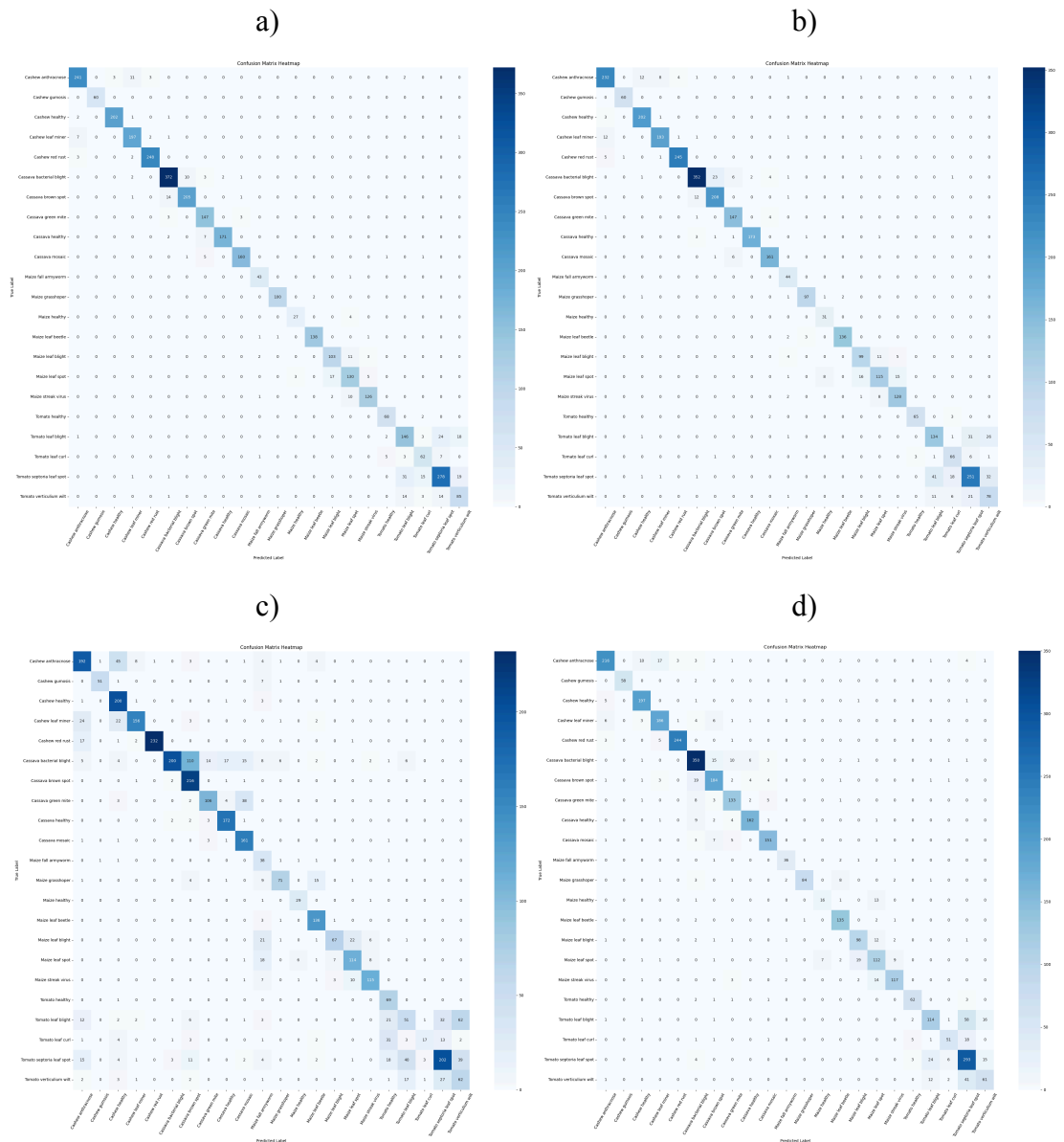
Gambar 14. Perbandingan grafik *training vs validation loss* dan *training vs validation accuracy* pada model DenseNet-121.

4. Hasil Pengujian

Classification Report:					Classification Report:				
	precision	recall	f1-score	support		precision	recall	f1-score	support
Cashew anthracnose	0.95	0.93	0.94	260	Cashew anthracnose	0.92	0.89	0.90	260
Cashew gumosis	1.00	1.00	1.00	60	Cashew gumosis	0.98	1.00	0.99	60
Cashew healthy	0.99	0.98	0.98	206	Cashew healthy	0.93	0.98	0.95	206
Cashew leaf miner	0.92	0.95	0.93	208	Cashew leaf miner	0.95	0.93	0.94	208
Cashew red rust	0.98	0.98	0.98	253	Cashew red rust	0.98	0.97	0.97	253
Cassava bacterial blight	0.94	0.95	0.95	390	Cassava bacterial blight	0.95	0.90	0.93	390
Cassava brown spot	0.95	0.93	0.94	221	Cassava brown spot	0.89	0.94	0.91	221
Cassava green mite	0.91	0.96	0.93	153	Cassava green mite	0.92	0.96	0.94	153
Cassava healthy	0.99	0.95	0.97	180	Cassava healthy	0.99	0.96	0.97	180
Cassava mosaic	0.97	0.95	0.96	168	Cassava mosaic	0.94	0.96	0.95	168
Maize fall armyworm	0.90	0.98	0.94	44	Maize fall armyworm	0.73	1.00	0.85	44
Maize grasshopper	0.99	0.98	0.99	102	Maize grasshopper	0.96	0.95	0.96	102
Maize healthy	0.90	0.87	0.89	31	Maize healthy	0.78	1.00	0.87	31
Maize leaf beetle	0.99	0.98	0.98	141	Maize leaf beetle	0.99	0.96	0.97	141
Maize leaf blight	0.84	0.87	0.85	119	Maize leaf blight	0.84	0.83	0.84	119
Maize leaf spot	0.83	0.84	0.84	155	Maize leaf spot	0.85	0.74	0.79	155
Maize streak virus	0.93	0.91	0.92	139	Maize streak virus	0.86	0.92	0.89	139
Tomato healthy	0.89	0.97	0.93	70	Tomato healthy	0.96	0.93	0.94	70
Tomato leaf blight	0.74	0.75	0.75	194	Tomato leaf blight	0.72	0.69	0.70	194
Tomato leaf curl	0.72	0.79	0.76	78	Tomato leaf curl	0.69	0.85	0.76	78
Tomato septoria leaf spot	0.86	0.81	0.83	345	Tomato septoria leaf spot	0.81	0.73	0.77	345
Tomato verticillium wilt	0.69	0.73	0.71	117	Tomato verticillium wilt	0.57	0.67	0.61	117
accuracy			0.91	3634	accuracy			0.89	3634
macro avg	0.90	0.91	0.91	3634	macro avg	0.87	0.90	0.88	3634
weighted avg	0.91	0.91	0.91	3634	weighted avg	0.89	0.89	0.89	3634

Classification Report:					Classification Report:				
	precision	recall	f1-score	support		precision	recall	f1-score	support
Cashew anthracnose	0.71	0.74	0.72	260	Cashew anthracnose	0.92	0.83	0.87	260
Cashew gumosis	0.96	0.85	0.90	60	Cashew gumosis	1.00	0.97	0.98	60
Cashew healthy	0.68	0.97	0.80	206	Cashew healthy	0.91	0.96	0.93	206
Cashew leaf miner	0.91	0.75	0.82	208	Cashew leaf miner	0.87	0.89	0.88	208
Cashew red rust	1.00	0.92	0.95	253	Cashew red rust	0.98	0.96	0.97	253
Cassava bacterial blight	0.96	0.51	0.67	390	Cassava bacterial blight	0.85	0.90	0.87	390
Cassava brown spot	0.60	0.98	0.74	221	Cassava brown spot	0.82	0.83	0.83	221
Cassava green mite	0.83	0.69	0.76	153	Cassava green mite	0.82	0.87	0.84	153
Cassava healthy	0.87	0.96	0.91	180	Cassava healthy	0.93	0.90	0.91	180
Cassava mosaic	0.73	0.96	0.83	168	Cassava mosaic	0.90	0.90	0.90	168
Maize fall armyworm	0.30	0.86	0.44	44	Maize fall armyworm	0.84	0.82	0.83	44
Maize grasshopper	0.86	0.70	0.77	102	Maize grasshopper	0.98	0.82	0.89	102
Maize healthy	0.78	0.94	0.85	31	Maize healthy	0.70	0.52	0.59	31
Maize leaf beetle	0.81	0.96	0.88	141	Maize leaf beetle	0.88	0.96	0.92	141
Maize leaf blight	0.86	0.56	0.68	119	Maize leaf blight	0.80	0.82	0.81	119
Maize leaf spot	0.77	0.74	0.75	155	Maize leaf spot	0.71	0.72	0.72	155
Maize streak virus	0.87	0.83	0.85	139	Maize streak virus	0.89	0.84	0.86	139
Tomato healthy	0.48	0.99	0.65	70	Tomato healthy	0.86	0.89	0.87	70
Tomato leaf blight	0.43	0.26	0.33	194	Tomato leaf blight	0.74	0.59	0.66	194
Tomato leaf curl	0.77	0.22	0.34	78	Tomato leaf curl	0.82	0.65	0.73	78
Tomato septoria leaf spot	0.74	0.59	0.65	345	Tomato septoria leaf spot	0.70	0.85	0.77	345
Tomato verticillium wilt	0.38	0.53	0.44	117	Tomato verticillium wilt	0.66	0.52	0.58	117
accuracy			0.73	3634	accuracy			0.84	3634
macro avg	0.74	0.75	0.72	3634	macro avg	0.84	0.82	0.83	3634
weighted avg	0.77	0.73	0.73	3634	weighted avg	0.84	0.84	0.84	3634

Gambar 15. *Classification reports* untuk: (a) EfficientNet-B0, (b) ResNet-50, (c) InceptionV3, dan (d) DenseNet-121.



Gambar 16. *Confusion matrix* untuk: (a) EfficientNet-B0, (b) ResNet-50, (c) InceptionV3, dan (d) DenseNet-121.

V. PEMBAHASAN DAN KETERBATASAN

1. Performa Analisis

Berdasarkan gambar 11, 12, 13, dan 14, serta tabel 6, hasil pada data *training*, *validation*, dan *testing* menunjukkan bahwa EfficientNet-B0 merupakan model yang paling stabil. Performanya naik dengan konsisten pada awal epoch, lalu mulai stagnan dan berfluktuasi pada kisaran 90–91% sejak epoch ke-7, yang mengindikasikan bahwa model telah mencapai titik optimal tanpa menunjukkan gejala *overfitting* yang signifikan. Pola dari grafik ResNet-50 kurang stabil dengan fluktuasi yang cukup jelas, namun nilai akurasinya tetap tinggi dan berada di atas 85% sejak awal epoch, sehingga masih mampu menyaingi performa EfficientNet-B0. Namun, proses *training* ResNet-50 memakan waktu yang relatif lama, yaitu 3 jam 5 menit hanya untuk mencapai epoch ke-8, dibandingkan dengan EfficientNet-B0 yang mampu menyelesaikan 20 epoch dalam waktu sekitar 4 jam 8 menit yang menunjukkan bahwa EfficientNet-B0 lebih efisien dari sisi waktu *training*.

Sementara itu, DenseNet-121 juga menunjukkan kenaikan akurasi yang stabil, tapi mulai stagnan pada epoch ke-10 sampai 16 dengan akurasi validasi di sekitar 84%. Hal tersebut menunjukkan bahwa model sedikit *overfitting* dan telah mencapai batas pembelajarannya. Sedangkan InceptionV3 memulai *training*-nya dengan akurasi di bawah 0.4 pada epoch pertama, lalu menunjukkan pola yang sangat tidak stabil dengan fluktuasi tajam pada grafik akurasi dan loss. Performa dari InceptionV3 juga paling rendah di antara yang lainnya, artinya model tersebut kurang mampu melakukan generalisasi dengan baik dan cenderung *overfitting*.

Berdasarkan hasil *classification report* pada gambar 15, terlihat bahwa EfficientNet-B0 memperoleh akurasi tertinggi pada *test set*, yaitu 0.91, diikuti oleh ResNet-50 dengan akurasi 0.89, DenseNet-121 sebesar 0.84, dan InceptionV3 sebesar 0.73. Hasil ini menunjukkan bahwa EfficientNet-B0 memiliki performa paling baik untuk diuji pada dataset asli atau *real-world cases*. Namun, berdasarkan *confusion matrix* pada gambar 16, keempat model tersebut menunjukkan kelemahan yang serupa di 4 kelas, yaitu *tomato leaf blight*, *tomato leaf curl*, *tomato septoria leaf spot*, dan *tomato verticillium wilt*. Hal ini juga ditunjukkan dari nilai precision, recall, dan F1-score keempat kelas tersebut yang lebih rendah dibanding kelas lainnya di seluruh

model. Kondisi ini terjadi karena karakteristik visual dari keempat kelas tersebut terlihat mirip. Gambar-gambarnya berupa daun tomat dengan pola bercak, warna, dan tekstur yang saling menyerupai sehingga menyulitkan model saat klasifikasi.

Secara keseluruhan, perbedaan performa keempat model dipengaruhi oleh kesesuaian arsitektur dengan ukuran dan karakteristik dari dataset *crop pest and disease*. EfficientNet-B0 menunjukkan performa terbaik karena dirancang dengan prinsip *compound scaling* yang memungkinkan mengekstraksi detail visual secara efisien yang terlihat dari nilai akurasi dan F1-score yang relatif stabil hampir di seluruh kelas. Pada ResNet-50, walaupun memiliki *skip connection* yang membantu stabilitas *training*, performanya mengalami fluktuasi dan efisiensi komputasinya lambat. DenseNet-121 memiliki kemampuan pembelajaran yang baik dengan aliran fitur yang padat, tapi mengalami stagnasi pada epoch akhir *training*. Sementara itu, performa dari model InceptionV3 merupakan yang paling rendah karena arsitekturnya yang lebih kompleks tidak dapat dimanfaatkan secara optimal oleh dataset.

2. Tantangan

- a) Dataset terdapat banyak gambar yang rusak dan duplikasi data, baik yang berada di dalam kelas yang sama maupun kelas yang berbeda. Dari 25.220 gambar awal, terdapat 94 gambar rusak yang harus dibuang dan 1.055 gambar terduplikat yang harus dibuang agar tidak mengganggu proses *training* dan hasil evaluasi model.
- b) Keterbatasan waktu komputasi yang menjadi kendala utama, khususnya pada model kompleks, seperti ResNet-50 yang membutuhkan waktu *training* cukup lama karena dilakukan tanpa GPU khusus, hanya menggunakan laptop pribadi. Kondisi ini membatasi eksplorasi lebih lanjut, seperti melakukan eksperimen dengan *hyperparameter tuning*.

3. Trade-Offs

Beberapa model seperti ResNet-50 dan InceptionV3 terhenti pada epoch yang lebih awal karena terkena callbacks EarlyStopping, sehingga belum mencapai performa optimal yang mungkin akan didapatkan jika dilanjutkan sampai epoch ke-20. Namun, hasil tersebut tetap diambil karena adanya keterbatasan *device* untuk training. Walaupun ResNet-50 dan InceptionV3 hanya berlangsung sampai epoch ke-8 dan ke-10, proses *training*-nya telah mencapai waktu sekitar 3 jam, dan berdasarkan grafik performa *accuracy* dan *loss*-nya juga telah menunjukkan kondisi

stagnan, sehingga hasil *training* lanjutannya diperkirakan tidak memberikan peningkatan yang signifikan.

VI. KESIMPULAN DAN *FUTURE WORK*

Penelitian ini membandingkan 4 arsitektur *pretrained model* CNN, yaitu EfficientNet-B0, ResNet-50, DenseNet-121, dan InceptionV3 yang di-*fine-tuning* untuk melakukan klasifikasi pada *dataset crop pest and disease*. Berdasarkan hasil eksperimen, EfficientNet-B0 menunjukkan performa terbaik dengan akurasi tertinggi pada *test set* sebesar 0.91, serta pola performa yang paling stabil pada data training dan validation. Namun, performa prediksi modelnya masih menunjukkan kelemahan pada 4 kelas di dataset, yaitu *tomato leaf blight*, *tomato leaf curl*, *tomato septoria leaf spot*, dan *tomato verticillium wilt*, yang ditandai dengan nilai F1-score yang lebih rendah dibandingkan kelas lainnya. Keempat kelas tersebut memiliki kemiripan visual yang tinggi, seperti pola bercak, warna, dan tekstur daun tomat yang serupa, sehingga model sering mengalami kesalahan dalam mengklasifikasi gambar tersebut di *real-world cases*.

Meskipun demikian, hasil yang diperoleh oleh EfficientNet-B0 tetap lebih baik dibandingkan ResNet-50, DenseNet-121, dan InceptionV3, karena arsitekturnya dirancang berdasarkan prinsip *compound scaling*, yang memungkinkan ekstraksi fitur yang efisien dan performanya juga stabil pada sebagian besar kelas. Untuk penelitian selanjutnya, karena EfficientNet-B0 telah terbukti sebagai model dengan performa terbaik, penelitian selanjutnya dapat diperluas dengan menggunakan varian dari EfficientNet (B1-B7) lainnya yang memiliki *compound scaling coefficient* yang lebih besar untuk mengeksplorasi pengaruh peningkatan *depth*, *width*, dan *resolution*-nya terhadap performa klasifikasi. Selain itu, penelitian selanjutnya juga bisa mencoba melakukan *hyperparameter tuning*, seperti mengatur *learning rate*, *batch size*, dan strategi regularisasi jika spesifikasi *device* yang digunakan untuk *training* memadai. Kedua hal tersebut bisa dilakukan untuk mendapatkan potensi performa model yang maksimal.

VII. DAFTAR PUSTAKA

- Deputy, K. V., Passi, K., & Jain, C. K. (2023). Crop disease detection using deep learning techniques on images. *Journal of Computer Science*, 19(12), 1438–1449. <https://doi.org/10.3844/jcssp.2023.1438.1449>
- Jeyabose, A., Eunice, R. J., Popescu, D. E., & Chowdary, K. (2022). Deep learning-based leaf disease detection in crops using images for agricultural applications. *Agronomy*, 12(10), Article 2359. <https://doi.org/10.3390/agronomy12102359>
- Niranjan, C. K., & Mallikarjuna, P. B. (2022). Insect pest image detection and classification using deep learning. *International Journal of Advanced Computer Science and Applications*, 13(9). <https://doi.org/10.14569/IJACSA.2022.0130947>
- Ramcharan, A., Baranowski, K., McCloskey, P. C., Ahmed, B., Legg, J. P., & Hughes, D. P. (2017). Deep learning for image-based cassava disease detection. *Frontiers in Plant Science*, 8, Article 1852. <https://doi.org/10.3389/fpls.2017.01852>
- Sankalana, N. (2021). *Crop pest and disease detection* [Dataset]. Kaggle. <https://www.kaggle.com/datasets/nirmalsankalana/crop-pest-and-disease-detection>
- Sharma, V., Tripathi, A. K., & Mittal, H. (2022). Technological advancements in automated crop pest and disease detection: A review and ongoing research. In *Proceedings of the 2022 International Conference on Computing, Communication, Security and Intelligent Systems (IC3SIS)* (pp. 1–6). IEEE. <https://doi.org/10.1109/IC3SIS54991.2022.9885605>
- Tan, M., & Le, Q. V. (2019). EfficientNet: Rethinking model scaling for convolutional neural networks. In *Proceedings of the 36th International Conference on Machine Learning (ICML)* (pp. 6105–6114). PMLR. <https://proceedings.mlr.press/v97/tan19a/tan19a.pdf>

VIII. LAMPIRAN

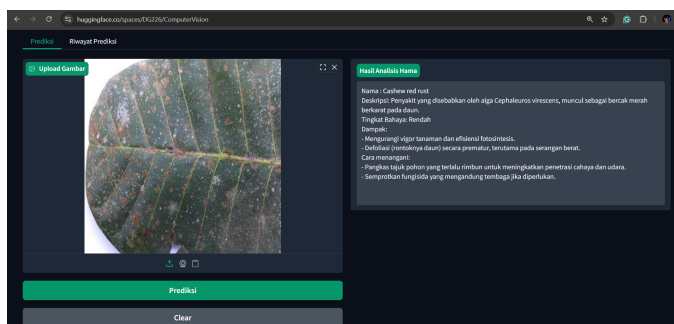
1. Pernyataan Kontribusi Anggota Tim

Team Member	David Goanli	Kelson	Rafael Komala	Meghan Hillary M.	Jason Emanuel H.
EDA		✓			
Preprocessing			✓		
Modelling	✓	✓	✓	✓	✓
Evaluation & Result	✓	✓	✓	✓	✓
Apps Deployment	✓				
Final Report		✓	✓	✓	✓
Demo Video	✓				

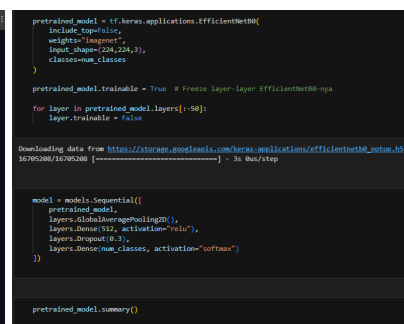
2. Dokumen Pendukung

No.	Dokumen	Link
1.	GitHub Repo	https://github.com/Luceatt/Final-Project-Computer-Vision
2.	Application	huggingface.co/spaces/DG226/ComputerVision
3.	Demo Video	Demo CV.mp4

3. Screenshots and Code Snippets



Gambar 17. User interface dari aplikasi PlantCare



Gambar 18. Contoh Code Snippet